

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Friday 11th September, 2026

Chapter 1

Experiments

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on the experimental validation of the method and its ability to consistently identify meaningful conceptual structures. The remainder of this chapter is organized as follows.

- Section 1.1 describes the software implementation of the CAR pipeline.
- Section 1.2 details the experimental configuration, including the environments, the reinforcement learning setup, and the training of the surrogate policies.
- Section 1.3 presents and analyses the results obtained by applying CAR and its metrics.
- Section ?? summarizes the findings and discusses their implications.

1.1 Software Implementation

Deep Learning (DL) and Reinforcement Learning (RL) are inherently empirical fields, in which the effectiveness of a method is assessed through experimentation. For this reason, particular attention is given to the implementation of CAR. It relies on established libraries that provide efficient and widely used implementations of its main components. The implementation is made publicly available as open-source to support the objective and reproducible replication of the reported results.¹

CAR is organized as a pipeline of five stages, shown in Figure 1.1. Each stage is an independent executable, and stages communicate only through artifacts written to disk. This decoupling allows any stage to be rerun or inspected independently. The parameters of each experiment are gathered in a single configuration object, described in Section 1.2. Every result can therefore be traced back to the settings that produced it.

Table 1.1 summarizes the main third-party libraries used at each stage of the pipeline.

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space

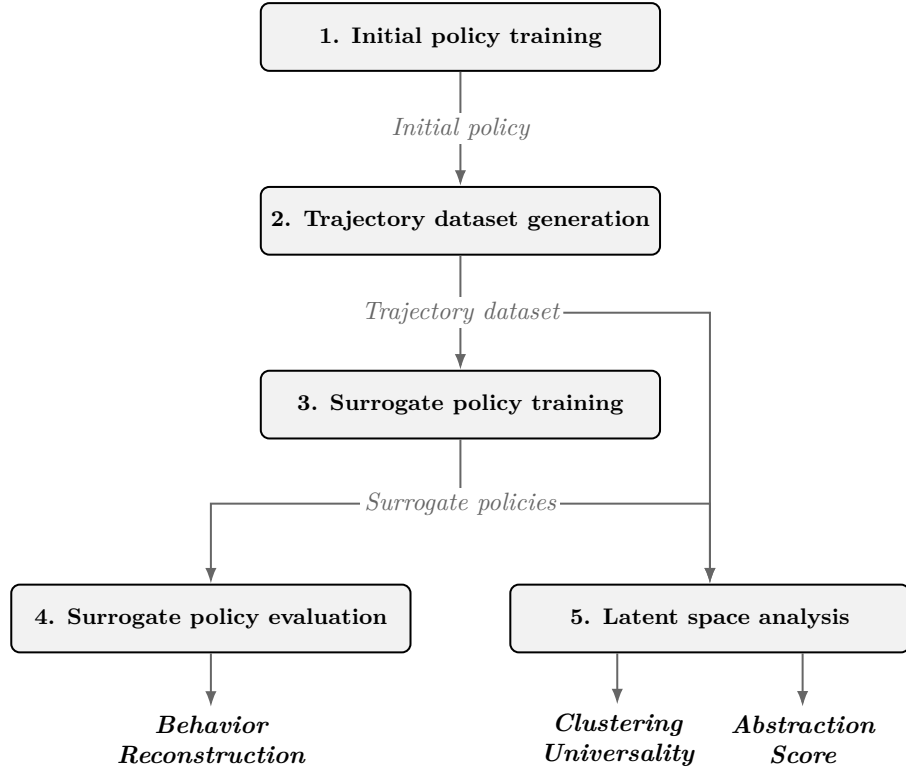


Figure 1.1: The CAR pipeline.

1.1.1 Initial policy training

The first stage trains the initial policy using RLlib [Moritz et al., 2018, Liang et al., 2018]. RLlib provides a highly scalable implementation of RL algorithms and supports a wide range of state-of-the-art methods. It supports distributed training, allowing the computational workload to be spread across multiple processes and machines.

The use of RLlib allows CAR to be applied to any environment implemented using the Gymnasium interface [Towers et al., 2024]. This provides a common interface for environments with different observation and action spaces. This flexibility allows CAR to be applied to a wide range of environments.

By default, the RL algorithm used for the initial policy is Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Other algorithms supported by RLlib, such as Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3, can also be selected through the same configuration.

The policy and critic networks are implemented using PyTorch [Paszke et al., 2019]. PyTorch provides efficient GPU computation and supports a wide range of neural network architectures. This provides a standardized implementation that is compatible with the broader DL ecosystem.

The output of this stage is a checkpoint containing the trained policy.

Pipeline Stage	Library	Description
Initial policy training	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed RL library used to train the initial policy.
	PyTorch [Paszke et al., 2019]	Deep learning framework implementing the policy and critic networks.
	Gymnasium [Towers et al., 2024]	Common interface used to plug in any RL environment.
Trajectory dataset generation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed environment runners used to collect trajectories in parallel.
	Apache Arrow [Apache Arrow, 2016]	Columnar file format (Parquet) used to store the trajectory datasets.
Surrogate policy training	PyTorch Lightning [Falcon, 2019]	Structured training loop used for the behavior cloning of the surrogate policies.
Surrogate policy evaluation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Wraps each surrogate policy as an RL policy to evaluate it in the environment.
Latent space analysis	cuML [RAPIDS, 2023]	GPU-accelerated clustering algorithms used to analyze the latent representations.

Table 1.1: Main software libraries used at each stage of the CAR pipeline.

1.1.2 Trajectory dataset generation

The second stage builds the dataset used to train the surrogate policies. The distributed execution capabilities of RLlib are used to collect trajectories in parallel. Multiple environment runners execute the initial policy simultaneously, distributing the computational workload across the available resources.

The initial policy is run in the environment, and the interactions between the agent and the environment are recorded. For each step, the dataset stores the observation together with the parameters of the action distribution produced by the policy. Trajectories are stored as Parquet files [Apache Arrow, 2016]. Parquet is a compressed columnar format widely used for large-scale data processing. Its columnar structure allows data to be read efficiently and supports concurrent access from multiple processes. This is particularly useful here because the datasets contain on the order of one million observation–action pairs and cannot be loaded entirely into memory.

1.1.3 Surrogate policy training

The third stage trains the surrogate policies using behavior cloning. This stage therefore converts the original RL problem into a supervised learning problem. The surrogate policies are trained to reproduce the action distributions generated by the initial policy .

The training procedure is implemented using PyTorch Lightning [Falcon, 2019]. PyTorch Lightning provides a structured interface for supervised training while remaining compatible with the PyTorch ecosystem. It handles common training operations such as data loading, optimization, checkpointing, and GPU execution, while allowing the underlying neural network implementation to remain fully controlled.

Once training is complete, the stage produces a set of multiple trained

79 surrogate policies.

80 1.1.4 Surrogate policy evaluation

81 The fourth stage measures how faithfully each surrogate policy reproduces the
 82 behavior of the initial policy. Each surrogate is wrapped so that it can be used
 83 as an RLlib policy and is then evaluated in the original environment. Its mean
 84 cumulative reward over multiple episodes is compared with that of initial policy.
 85 Evaluation episodes are collected using parallel environment runners.

86 This stage provides the value of the *Behavior Reconstruction*. It therefore
 87 verifies whether the surrogate policies adequately reproduce the behavior of the
 88 initial policy before their latent representations are analyzed. If a surrogate does
 89 not reproduce the original behavior sufficiently well, its training can be repeated
 90 before proceeding to the next stage.

91 1.1.5 Latent space analysis

92 The fifth stage performs the main analysis of CAR. The latent representations
 93 produced by the surrogate policies form the point clouds that are analyzed using
 94 clustering algorithms.

95 The clustering step relies on the cuML library [RAPIDS, 2023], which pro-
 96 vides GPU-accelerated implementations of several clustering algorithms. This
 97 significantly reduces the computational cost of clustering and makes it possible
 98 to analyze point clouds containing tens of thousands of representations.

99 The resulting partitions are used to compute the *Clustering Universality* and
 100 the *Abstraction Score*. The outputs of this stage are the resulting metric values
 101 and the material required for the qualitative analysis presented in the following
 102 sections.

103 1.2 Experimental Configuration

104 This section describes the setup shared by all the experiments reported in this
 105 chapter: the environments, the neural architecture of the policies, and the main
 106 parameters of each pipeline stage. For every environment, a single initial policy
 107 is trained and then imitated by $n = 3$ surrogate policies.

108 1.2.1 Environments

109 All experiments use environments from the Atari benchmark [Bellemare et al., 2013],
 110 accessed through the Arcade Learning Environment and configured following
 111 common practice in reinforcement learning research [Machado et al., 2017]. In
 112 every environment the agent receives visual observations and selects actions from
 113 a discrete set. The six environments are shown in Figure 1.2.

114 **Breakout.** The agent moves a paddle at the bottom of the screen (Figure 1.2a)
 115 to bounce a ball into a wall of bricks, each broken brick scoring a point, and the
 116 ball speeding up as the wall is cleared. The observation is dominated by the
 117 layout of the remaining bricks, which changes gradually over a game.

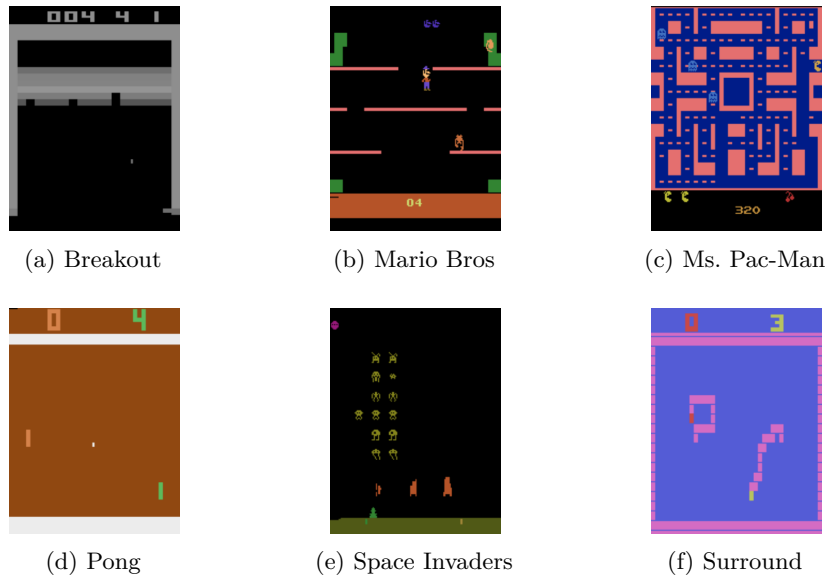


Figure 1.2: The six Atari environments used to evaluate CAR, shown as rendered game frames. The agent does not receive these frames directly: each observation is a stack of four consecutive frames, resized to 64×64 pixels and converted to grayscale, except for Ms. Pac-Man, which is kept in color.

118 **Mario Bros.** A single-screen platform game (Figure 1.2b). The player clears
 119 each phase by hitting enemies from below to flip them, then kicking them off
 120 the platforms before they recover. A central POW block flips every enemy at
 121 once but can only be used a few times per phase. Deciding when to spend the
 122 POW block is a clear strategic choice, which makes this environment useful for
 123 testing whether CAR recovers strategy-level distinctions.

124 **Ms. Pac-Man.** The player moves through a maze (Figure 1.2c), eating pellets
 125 while avoiding four ghosts, with power pellets briefly making the ghosts edible.
 126 The maze structure and the many possible local situations, such as a nearby
 127 ghost, an intersection, or an active power pellet, produce a large number of
 128 distinguishable states. This makes Ms. Pac-Man well suited to examining CAR
 129 at fine cluster granularities. It is also the only environment kept in color.

130 **Pong.** A minimal two-player game (Figure 1.2d) in which the agent controls
 131 one paddle and must send the ball past the opponent. The observation contains
 132 almost nothing beyond the two paddles, the ball, and the score. Pong therefore
 133 acts as a lower bound on how much structure CAR can be expected to recover.

134 **Space Invaders.** The agent moves a cannon along the bottom of the screen
 135 (Figure 1.2e), firing at descending rows of aliens while dodging their shots and
 136 using destructible shields, with a bonus saucer occasionally crossing the top. A
 137 game goes through visually distinct phases, from the full alien formation to a
 138 few fast aliens near the ground, which makes this environment convenient for
 139 illustrating the clusters found by CAR.

Surround. A two-player game on a grid (Figure 1.2f). Each player leaves a solid trail behind a moving cursor, and the goal is to force the opponent to crash into a trail or a wall. Every move permanently blocks a cell, so the state of a game is essentially the pattern of trails drawn so far, which grows continuously over an episode.

1.2.2 Neural architecture

The initial policy and the surrogate policies are convolutional networks that share the same feature-extraction trunk. The trunk is a sequence of four convolutional layers, with 16, 32, 64, and 128 channels, 4×4 kernels, and a stride of 2, each followed by layer normalization and a LeakyReLU activation. Its input is the observation, a stack of four consecutive 64×64 frames.

The initial policy follows an actor-critic design: the trunk feeds two separate multilayer perceptrons, one producing the parameters of the action distribution and one producing the value estimate. The surrogate policy has the encoder-decoder structure introduced in Section ??: its encoder f_{enc} is a trunk with the same architecture, and its decoder is a single multilayer perceptron producing the parameters of the action distribution. The surrogate has no value head, since it is trained only to reproduce the action distribution of the initial policy.

The latent space analyzed by CAR is the representation produced by the last convolutional layer of the surrogate encoder.

1.2.3 Parameters

Table 1.2 lists the parameters common to all experiments, grouped by pipeline stage. Parameters that vary across environments, such as the reward scaling and the exact number of training epochs, are given in the released configuration files.

1.3 Results

This section reports the results of applying CAR to the six environments described in Section 1.2. Sections 1.3.1 and 1.3.2 first check that the two learning stages of the pipeline behave as expected: that the initial policies reach a competent level of play, and that the surrogate policies faithfully imitate them. Section 1.3.3 then presents the analysis of the latent representations and the values of the three evaluation metrics.

1.3.1 Initial policy training

Figure 1.3 shows how the initial policies improve during PPO training. For each environment, the mean episode return is normalized so that 0% and 100% correspond to the lowest and highest mean return observed during that run. In every environment the return increases and then reaches a plateau, at which point training is stopped. The number of iterations required to reach this plateau varies from a few hundred, for Pong, to several thousand, for Surround and Ms. Pac-Man. For Ms. Pac-Man the return decreases after its peak; since the best checkpoint is retained rather than the last one, this does not affect the policy that is subsequently analyzed.

Observations	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Color	grayscale (color for Ms. Pac-Man)
Initial policy training (PPO)	
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training (behavior cloning)	
Surrogate policies per environment	3
Observation–action pairs per dataset	$\geq 10^6$
Optimizer	Adam
Learning rate	1×10^{-4}
Batch size	2048
Loss	MSE between action-distribution parameters
Train / validation split	80 / 20
Latent space analysis	
Observations per surrogate policy	50 000
Clustering algorithm	k -means, Euclidean distance, 10 restarts
Number of clusters K	2 to 30
Points for the 2D projection	10 000

Table 1.2: Parameters common to all experiments, grouped by pipeline stage.

181 To confirm that the trained policies play competently, their scores are com-
182 pared with the reference scores reported by [Machado et al., 2017]. Each initial
183 policy reaches a score of the same order as these references, so it constitutes a
184 reasonable target for imitation.

185 1.3.2 Surrogate policy training

186 Figure 1.4 shows, for each environment, the training action loss of the three
187 surrogate policies: the mean squared error between the action-distribution
188 parameters of the surrogate and those of the initial policy. In every case the loss
189 decreases smoothly, and the three independently trained policies follow almost
190 identical trajectories, converging to the same value; the three curves of each

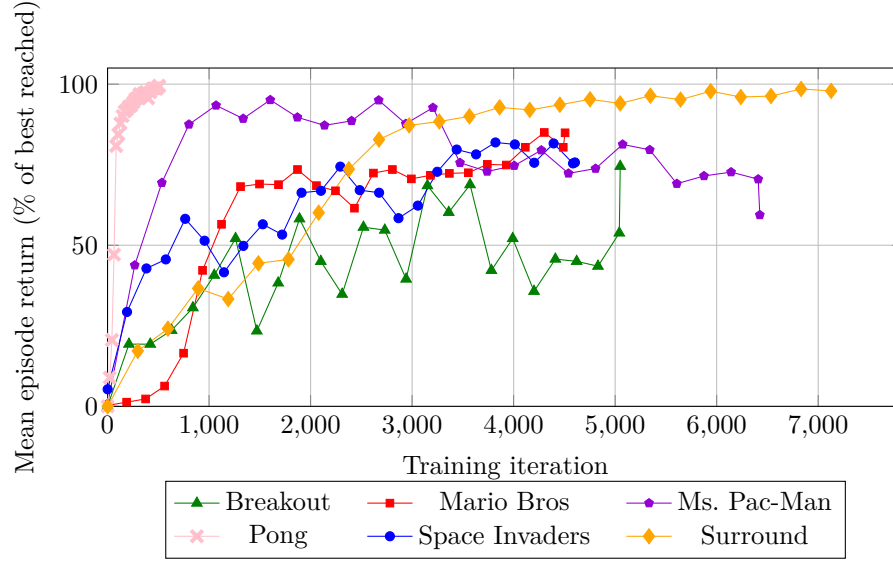


Figure 1.3: Training of the initial policies with PPO, one curve per environment.

environment largely overlap. This consistency matters for CAR: the method compares the latent representations of independently trained policies, and this comparison is only meaningful if these policies implement the same decision function. The behavioral fidelity of the surrogate policies is quantified by the *Behavior Reconstruction* in the next section.

1.3.3 Latent space analysis

We first evaluate the ability of the surrogate policies to replicate the behavior of the initial policies (*Behavior Reconstruction*, Table 1.3). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for Breakout to 1.00 for Ms. Pac-Man and Surround. They therefore reproduce the decisions of the initial policies closely enough for their latent representations to be analyzed in place of those of the initial policies.

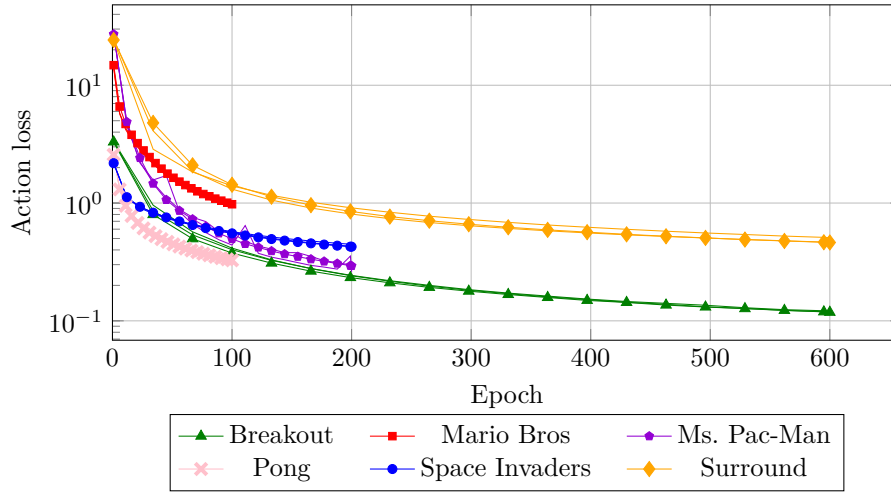


Figure 1.4: Training action loss of the surrogate policies, on a logarithmic scale, three curves per environment.

Environment	<i>Behavior Reconstruction</i>	<i>Abstraction Score</i>	<i>Clustering Universality</i>
Breakout	0.83 ± 0.03	0.82 ± 0.07	0.48 ± 0.11
Mario Bros	0.99 ± 0.03	0.71 ± 0.14	0.55 ± 0.10
Ms. Pac-Man	1.00 ± 0.03	0.43 ± 0.09	0.65 ± 0.08
Pong	0.95 ± 0.01	0.99 ± 0.00	0.49 ± 0.05
Space Invaders	0.97 ± 0.07	0.69 ± 0.06	0.67 ± 0.07
Surround	1.00 ± 0.01	0.55 ± 0.07	0.61 ± 0.09

Table 1.3: Evaluation of *Behavior Reconstruction*, *Abstraction Score* and *Clustering Universality* across the six Atari environments.

For the remainder of the analysis, a random subset of 50,000 observations is embedded into the Latent Space (LS) of each surrogate policy, using the representation extracted directly after the last convolutional layer, and the resulting point cloud $\mathcal{E}$ is partitioned with k -means clustering [MacQueen, 1967].

Figure 1.5 reports the evolution of *Clustering Universality* with respect to the number of clusters. After the first few values of K , every curve settles onto a plateau whose level depends on the environment, from about 0.45–0.50 for Breakout and Pong to about 0.65 for Ms. Pac-Man and Space Invaders. Since *Clustering Universality* is computed from the Adjusted Rand Index (ARI), and an ARI of 0.5 already corresponds to substantial agreement [Amezquita et al., 2020], these values indicate that independently trained surrogate policies recover a similar cluster structure in every environment, and that this structure is stable with respect to the number of clusters. The strength of this agreement varies across environments, but remains clearly above chance everywhere. The absence of a single optimal number of clusters suggests that K should be chosen according to the level of granularity desired for the analysis.

The *Abstraction Score*, in contrast, varies markedly across environments. It is close to 1 for Pong (0.99) and high for Breakout, Mario Bros and Space Invaders (0.69 to 0.82): in these environments the latent clusters are largely independent of a direct clustering of the observations or of the action distributions. It is lower for Surround (0.55) and Ms. Pac-Man (0.43). In these two environments the position of the agent on the screen largely determines both the raw observation and the decision to take, so a clustering of the LS naturally coincides with a clustering of the observations, and the *Abstraction Score* cannot separate the two; a low score does not by itself imply that the representation is not abstract.

With fewer clusters, CAR highlights broad strategic patterns. For instance, in the Mario Bros environment, the two main groups correspond to whether the POW block is present (highlighted in red in Figure 1.7A) or not. This is particularly interesting, as the use of the POW block is a key determinant of strategy in this environment [AtariAge, 2024].

With a larger number of clusters, the analysis captures more fine-grained tactical behaviors, such as “Attacking the left” and “Navigating through the level” (see Figure 1.7B).

Both perspectives are valid, depending on whether the goal is to study strategic or tactical behaviors. Similar interpretable structures can be observed across the evaluated environments.

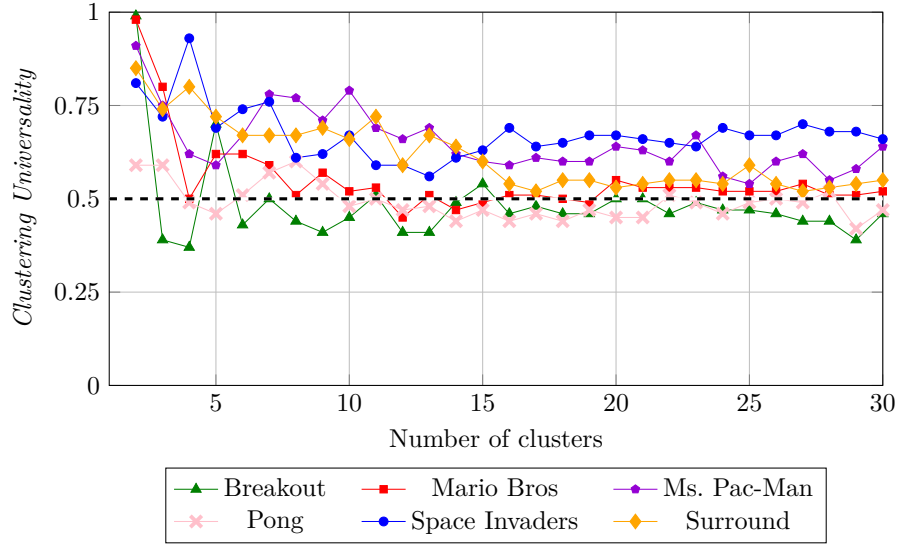


Figure 1.5: Evolution of *Clustering Universality* with respect to the number of clusters, for each of the six Atari environments.

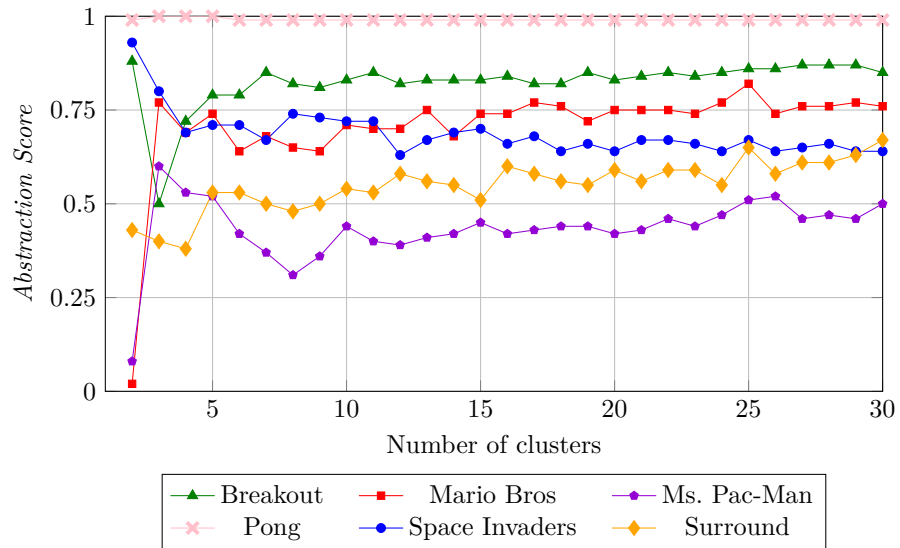
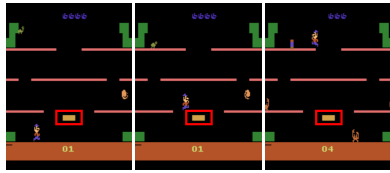
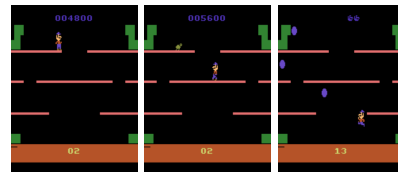


Figure 1.6: Evolution of *Abstraction Score* with respect to the number of clusters, for each of the six Atari environments.

(A) The case with 2 clusters.



The POW block is present.

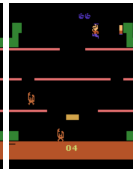


The POW block is absent.

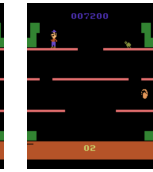
(B) The case with 5 clusters.



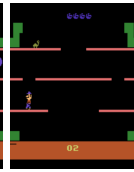
The POW block is present.



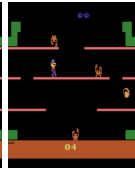
Move on the left platform.



Navigating through the level.



Attacking the left.



Attacking the right.



Figure 1.7: Effect of the number of clusters on the groupings identified by CAR in the Mario Bros environment.

Acronyms

240	APPO Asynchronous Proximal Policy Optimization 2
241	ARI Adjusted Rand Index 10
242	CAR Clustering Agent Representation 1–6, 8, 10, 12
243	DL Deep Learning 1, 2
244	DQN Deep Q-Networks 2
245	GPU Graphics Processing Unit 3
246	LS Latent Space 10
247	PPO Proximal Policy Optimization 2, 6, 8
248	RL Reinforcement Learning 1–3

Bibliography

- [Amezquita et al., 2020] Amezquita, R. A., Lun, A. T., Becht, E., Carey, V. J., Carpp, L. N., Geistlinger, L., Marini, F., Rue-Albrecht, K., Risso, D., Soneson, C., et al. (2020). Orchestrating single-cell analysis with Bioconductor, volume 17. Nature Publishing Group US New York. **Book**.
- [Apache Arrow, 2016] Apache Arrow (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- [AtariAge, 2024] AtariAge (2024). Mario bros. atari 2600. **Web**.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article**.
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon, 2019] Falcon, W. (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web**.
- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article**.
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.

- 282 [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R.,
283 Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018).
284 Ray: A distributed framework for emerging AI applications. In 13th USENIX
285 Symposium on Operating Systems Design and Implementation (OSDI), pages
286 561–577.
- 287 [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J.,
288 Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison,
289 A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy,
290 S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An
291 imperative style, high-performance deep learning library. In Advances in
292 Neural Information Processing Systems (NeurIPS).
- 293 [RAPIDS, 2023] RAPIDS (2023). RAPIDS cuML: GPU-accelerated machine
294 learning. <https://github.com/rapidsai/cuml>.
- 295 [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and
296 Klimov, O. (2017). Proximal policy optimization algorithms.
- 297 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
298 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
299 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
300 environments.